応用情報技術者試験　テクノロジ系　No.12

システム開発技術

システム要件定義・ソフトウェア要件定義・設計・実装・構築・統合・テスト・導入・受入れ支援・保守・廃棄

目次

1. 12-1 システム要件定義・ソフトウェア要件定義（21語）
2. 12-2 設計（33語）
3. 12-3 実装・構築（12語）
4. 12-4 統合・テスト（23語）
5. 12-5 導入・受入れ支援（10語）
6. 12-6 保守・廃棄（13語）
7. まとめ

12-1 システム要件定義・ソフトウェア要件定義 要件・要件を集めて決める（発展）

機能要件・非機能要件
Functional / Non-functional Requirements

機能要件は何ができるか、非機能要件は性能・信頼性・セキュリティなどどれだけうまくできるか

非機能要求グレード
Non-functional Requirements Grade

非機能要件を、可用性・性能・運用・移行・セキュリティなどの項目で段階的に決めるための表（IPA）

ユースケース
Use Case

利用者（アクタ）とシステムのやり取りを、目的ごとに書いたもの

CRUD分析
CRUD Analysis

処理ごとにデータを Create・Read・Update・Delete のどれで使うかを表にして整理する方法

要求と要件
Requirement

利用者が望むこと（要求）と、それを整理してシステムで実現すると決めたこと（要件）

業務要件
Business Requirements

業務として何を実現したいかをまとめたもの

ポイント システム要件のもとになる

12-1 システム要件定義・ソフトウェア要件定義 要件を集めて決める（発展）

システム要件定義
System Requirements Definition

システム全体に求める機能・性能・運用などを決めること

ソフトウェア要件定義
Software Requirements Definition

システム要件のうち、ソフトウェアが受け持つ部分の要件を決めること

ヒアリング・インタビュー
Interview

利用者に直接話を聞いて要求を集める方法

アンケート
Questionnaire

多くの人から、決めた質問で要求を集める方法

観察法
Observation

実際の仕事の様子を見て、要求を見つける方法

ワークショップ
JAD / Joint Application Development

利用者と開発者が集まって、一緒に要件を決める会

Joint＝共同の／Application＝アプリケーション／Development＝開発

12-1 システム要件定義・ソフトウェア要件定義 要件を集めて決める（発展）

要件の優先順位付け
MoSCoW分析

要件を「必須・望ましい・あれば良い・今回はしない」に分ける方法

ポイント Must・Should・Could・Won't

要件のトレーサビリティ
Requirements Traceability

要件が、設計・プログラム・テストのどこで実現されたかをたどれること

ステークホルダ
Stakeholder

システムに関わり、影響を受ける人や組織

ポイント 利用者・発注者・運用者など

業務フロー図
Business Flow Diagram

仕事の流れと担当を、図で表したもの

ポイント BPMNという書き方の標準がある

BPMN
びーぴーえむえぬ / Business Process Model and Notation

業務の流れを図で表すための書き方の標準

Business Process＝業務の流れ／Model＝モデル／Notation＝表記

性能要件
Performance Requirements

応答時間や処理件数など、どれくらい速く多くさばけるかの要件

12-1 システム要件定義・ソフトウェア要件定義 要件を集めて決める（発展）

可用性要件
Availability Requirements

稼働率や止まってよい時間など、どれくらい止まらないかの要件

移行要件
Migration Requirements

古いシステムからデータや業務をどう移すかの要件

運用・保守性要件
Operation / Maintainability Requirements

監視・バックアップ・保守のしやすさなどの要件

12-2 設計 まとめ表

モジュールの強さと結合		
尺度	最もよい	最も悪い
モジュール強度（凝集度）	機能的強度	暗合的強度
モジュール結合度	データ結合	内容結合

12-2 設計 設計の手法

DFD
でぃーえふでぃー / Data Flow Diagram

データの流れを、プロセス・データストア・外部実体・データフローで表す図

ポイント 処理の順番や条件は表さない

Data＝データ／Flow＝流れ／Diagram＝図

UMLの主な図
UML Diagrams

クラス図（構造）、シーケンス図（時間順のやり取り）、コミュニケーション図（オブジェクトのつながり）、アクティビティ図（処理の流れ）、状態マシン図（状態の変化）

汎化・集約
Generalization / Aggregation

汎化は「is-a」の関係（上位クラスと下位クラス）、集約は「part-of」の関係（全体と部分）

多態性
ポリモーフィズム / Polymorphism

同じメッセージでも、受け取るオブジェクトによって違う動きをすること

デザインパターン
Design Pattern

オブジェクト指向設計でよく使う設計の型

例 Singleton、Observer、Factory Method

モジュール強度
Module Strength

モジュールの中の機能のまとまりの強さ

ポイント 機能的強度が最も強くよい

12-2 設計 設計の手法・設計の手法（発展）

モジュール結合度
Module Coupling

モジュールどうしのつながりの強さ

ポイント データ結合が最も弱くよい

決定表
デシジョンテーブル / Decision Table

条件の組合せと、それに対する動作を表で表したもの

レビュー手法
Review

ウォークスルー（作成者が説明）、インスペクション（モデレータが進行し役割を決める）、ラウンドロビン（参加者が順に進行）

外部設計・内部設計
External / Internal Design

利用者から見える画面や帳票などを決める設計と、プログラムの中の作りを決める設計

方式設計
Architecture Design

システムの構成（ハードウェア・ソフトウェア・ネットワーク）を決める設計

画面設計・帳票設計
Screen / Report Design

画面や印刷物に何をどこに出すかを決める設計

12-2 設計 設計の手法（発展）

コード設計
Code Design

商品番号などのコードの付け方を決める設計

ポイント 桁数や体系、チェックディジットを決める

チェックディジット
Check Digit

コードの最後に付け、入力の誤りを見つけるための数字

ポイント 決まった計算で求める

構造化分析
Structured Analysis

業務をデータの流れで捉え、DFDを使って分析する方法

オブジェクト指向設計
Object-oriented Design

データと処理をまとめたオブジェクトの組合せでシステムを考える設計

クラス・インスタンス
Class / Instance

オブジェクトのひな形と、ひな形から作った実際のオブジェクト

継承
けいしょう / Inheritance

上位クラスの性質を下位クラスが引き継ぐこと

12-2 設計 設計の手法（発展）

カプセル化
Encapsulation

データとそれを扱う処理をまとめ、中身を外から直接さわれないようにすること

ポイント 情報隠蔽

オーバーライド・オーバーロード
Override / Overload

上位クラスの処理を下位クラスで書き換えることと、同じ名前で引数の違う処理を複数作ること

クラス図
Class Diagram

クラスの属性・操作と、クラスどうしの関係を表すUMLの図

シーケンス図
Sequence Diagram

オブジェクトどうしのメッセージのやり取りを時間の順に表すUMLの図

アクティビティ図
Activity Diagram

処理の流れや分岐、並行する作業を表すUMLの図

ステートマシン図
State Machine Diagram

状態と、状態が変わるきっかけを表すUMLの図

12-2 設計 設計の手法（発展）

コミュニケーション図
Communication Diagram

オブジェクトどうしのつながりとメッセージを表すUMLの図

ポイント 時間の順より、つながりに注目する

配置図・コンポーネント図
Deployment / Component Diagram

機器へのソフトウェアの置き方と、部品の組み立てを表すUMLの図

ユースケース記述
Use Case Description

ユースケースの流れを、手順や例外の場合も含めて文章で書いたもの

STS分割・TR分割
STS / TR Partitioning

データの流れを入力・変換・出力に分ける方法と、取引の種類ごとに分ける方法

ポイント 構造化設計のモジュール分割

凝集度の種類
Types of Cohesion

機能的・情報的・連絡的・手順的・時間的・論理的・暗合的の順に、まとまりが弱くなる

結合度の種類
Types of Coupling

データ・スタンプ・制御・外部・共通・内容の順に、依存が強くなる

ポイント データ結合が最もよい

12-2 設計 設計の手法（発展）

MVCモデル
Model-View-Controller

処理を、データ（Model）、画面（View）、操作の受付（Controller）に分ける作り方

シングルトン・オブザーバ
Singleton / Observer Pattern

インスタンスを1つだけにするパターンと、変化を登録した相手に知らせるパターン

ポイント デザインパターンの例

状態遷移表
State Transition Table

状態とイベントの組合せで、次の状態と動作を表にしたもの

12-3 実装・構築 実装・作ってたしかめる（発展）

コーディング規約
Coding Standards

プログラムの書き方の決まり

ポイント 読みやすさと品質をそろえる

静的解析
Static Analysis

プログラムを動かさずにソースコードを調べて問題を見つけること

ペアプログラミング
Pair Programming

2人1組で、1人が書き、もう1人が確かめながら作ること

テスト駆動開発
TDD / Test-Driven Development

先にテストを書き、それに通るようにプログラムを作る進め方

インスペクション
Inspection

進行役（モデレータ）を立て、役割と手順を決めて行う公式なレビュー

ウォークスルー
Walkthrough

作成者が中心になって、関係者と成果物を順にたどって誤りを探すレビュー

12-3 実装・構築 作ってたしかめる（発展）

ラウンドロビン
Round Robin

参加者が順に進行役になって行うレビュー

机上チェック
Desk Check

プログラムを動かさずに、紙の上などで処理を追って誤りを探すこと

デバッグ
Debugging

プログラムの誤り（バグ）を見つけて直すこと

デバッガ
Debugger

プログラムを1行ずつ動かしたり、止めて変数を見たりする道具

コードカバレッジ
Code Coverage

テストでプログラムのどれだけの部分を通ったかの割合

単体テストの自動化
Unit Test Automation

テスト用のプログラムで、単体テストを何度でも自動で実行できるようにすること

例 JUnit

12-4 統合・テスト まとめ表

網羅の基準	
基準	**満たす条件**
命令網羅	すべての命令を1回は実行する
判定条件網羅（分岐網羅）	すべての分岐で真と偽を通る
条件網羅	判定の中の各条件が真と偽をとる
複数条件網羅	条件の組合せをすべて試す（最も強い）

12-4 統合・テスト テスト

ブラックボックステスト
Black Box Test

内部を見ず、仕様どおりに入出力が合うかを調べる

ポイント 同値分割・限界値分析・原因結果グラフ

ホワイトボックステスト
White Box Test

内部の構造を見て、命令や分岐を通るか調べる

ポイント 命令網羅・判定条件網羅・条件網羅・複数条件網羅

限界値分析
Boundary Value Analysis

有効な範囲の境目と、そのすぐ外の値をテストする方法

スタブ・ドライバ
Stub / Driver

スタブは下位モジュールの代わり（トップダウンテスト）、ドライバは上位モジュールの代わり（ボトムアップテスト）

回帰テスト
Regression Test

修正によってほかの部分に不具合が出ていないかを確かめるテスト

信頼度成長曲線
ゴンペルツ曲線 / Reliability Growth Curve

テストの時間と発見したバグの累計の関係を表すS字の曲線

ポイント 増え方が止まれば品質が安定してきたとみなす

12-4 統合・テスト テスト・テストの方法（発展）

エラー埋込み法
Error Seeding

わざと既知のエラーを埋め込み、その見つかった割合から残っているバグの数を推定する方法

テストケース
Test Case

入力・手順・期待する結果を組にしたテストの単位

同値分割
Equivalence Partitioning

入力を、同じように処理されるグループに分け、各グループから代表の値を選ぶ方法

原因結果グラフ
Cause-Effect Graph

入力の条件（原因）と結果の関係を図にし、テストケースを作る方法

実験計画法
Design of Experiments

直交表を使って、少ない回数で組合せを効率よくテストする方法

命令網羅
Statement Coverage

すべての命令を少なくとも1回は通るようにテストする基準

12-4 統合・テスト テストの方法（発展）

判定条件網羅
分岐網羅 / Branch Coverage

すべての分岐で、真と偽の両方を少なくとも1回は通るようにテストする基準

条件網羅
Condition Coverage

分岐の中の個々の条件が、真と偽の両方になるようにテストする基準

複数条件網羅
Multiple Condition Coverage

分岐の中の条件の真偽のすべての組合せをテストする基準

ポイント 最も厳しい

トップダウンテスト・ボトムアップテスト
Top-down / Bottom-up Test

上位のモジュールから順に結合する方法と、下位から順に結合する方法

ポイント 上からはスタブ、下からはドライバを使う

ビッグバンテスト
Big Bang Test

すべてのモジュールを一度に結合してテストする方法

ポイント 不具合の場所を見つけにくい

サンドイッチテスト
Sandwich Test

上からと下からの結合を同時に進めるテスト

12-4 統合・テスト テストの方法（発展）

システムテスト
System Test

システム全体が要件どおりに動くかを確かめるテスト

ポイント 機能・性能・負荷・セキュリティなど

負荷テスト・性能テスト
Load / Performance Test

たくさんの利用があっても、決めた速さで動くかを確かめるテスト

障害回復テスト
Recovery Test

障害が起きたあと、正しく元に戻せるかを確かめるテスト

ユーザビリティテスト
Usability Test

利用者が使いやすいかを、実際に使ってもらって確かめるテスト

探索的テスト
Exploratory Test

決まった手順を作らず、試しながら次に何を確かめるか考えて進めるテスト

12-5 導入・受入れ支援 導入・導入と移行（発展）

受入れテスト
Acceptance Test

発注者が、要件を満たしているかを確かめて受け入れるテスト

移行方式
Migration

一斉移行（一度に切り替え）、段階移行（少しずつ）、並行運用（新旧を同時に動かす）、パイロット移行（一部の部門で先に試す）

利用者マニュアル・教育
User Manual / Training

新しいシステムの使い方を文書と研修で伝えること

一斉移行
Big Bang Migration

ある日にすべてを新しいシステムに切り替える方式

ポイント 移行の期間は短いが、問題が出たときの影響が大きい

段階移行
Phased Migration

業務や拠点ごとに少しずつ切り替える方式

並行運用
Parallel Operation

しばらく古いシステムと新しいシステムを両方動かし、結果を比べながら移す方式

ポイント 安全だが手間とコストがかかる

12-5 導入・受入れ支援 導入と移行（発展）

パイロット移行
Pilot Migration

一部の部門で先に試してから、全体を切り替える方式

移行リハーサル
Migration Rehearsal

本番の前に、移行の手順を通して練習すること

検収
Acceptance Inspection

納品された物が契約どおりかを確かめて受け取ること

運用テスト
Operational Test

本番に近い環境で、実際の業務の流れで使えるかを確かめるテスト

12-6 保守・廃棄 保守・保守の種類（発展）

保守の種類
Types of Maintenance

是正保守（不具合を直す）、予防保守（問題が起きる前に直す）、適応保守（環境の変化に合わせる）、完全化保守（性能や保守性を上げる）

リファクタリング
Refactoring

外から見た動作は変えずに、中の構造を整理すること

是正保守
Corrective Maintenance

見つかった不具合を直す保守

リバースエンジニアリング
Reverse Engineering

既存のプログラムから仕様や設計を明らかにすること

システムの廃棄
Disposal

使わなくなったシステムを止め、データを確実に消去して処分すること

予防保守
Preventive Maintenance

不具合が表に出る前に、見つけて直しておく保守

12-6 保守・廃棄 保守の種類（発展）

適応保守
Adaptive Maintenance

OSや法律などの環境の変化に合わせてソフトウェアを変える保守

完全化保守
Perfective Maintenance

性能や使いやすさを良くするための保守

緊急保守
Emergency Maintenance

すぐに対応しないと業務が止まるときの、計画外の保守

保守性
Maintainability

直しやすさ・調べやすさ・変えやすさの度合い

レガシーシステム
Legacy System

古い技術で作られ、中身がわかりにくくなったシステム

ポイント 作り直しや移行が課題になる

マイグレーション
Migration

古いシステムのソフトウェアやデータを、新しい環境に移すこと

12-6 保守・廃棄 保守の種類（発展）

データ消去
Data Erasure

廃棄する機器から、データを読み出せないように確実に消すこと

ポイント 上書き・磁気消去・物理的な破壊

まとめ（1/3） この分野で覚える用語

小分類	用語
12-1 システム要件定義・ソフトウェア要件定義	機能要件・非機能要件・非機能要求グレード・ユースケース・CRUD分析・要求と要件・業務要件・システム要件定義・ソフトウェア要件定義・ヒアリング・インタビュー・アンケート・観察法・ワークショップ・要件の優先順位付け・要件のトレーサビリティ・ステークホルダ・業務フロー図・BPMN・性能要件・可用性要件・移行要件・運用・保守性要件
12-2 設計	DFD・UMLの主な図・汎化・集約・多態性・デザインパターン・モジュール強度・モジュール結合度・決定表・レビュー手法・外部設計・内部設計・方式設計・画面設計・帳票設計・コード設計・チェックディジット・構造化分析・オブジェクト指向設計・クラス・インスタンス・継承・カプセル化・オーバーライド・オーバーロード・クラス図・シーケンス図・アクティビティ図・ステートマシン図・コミュニケーション図・配置図・コンポーネント図・ユースケース記述・STS分割・TR分割・凝集度の種類・結合度の種類・MVCモデル・シングルトン・オブザーバ・状態遷移表

まとめ（2/3） この分野で覚える用語

小分類	用語
12-3 実装・構築	コーディング規約・静的解析・ペアプログラミング・テスト駆動開発・インスペクション・ウォークスルー・ラウンドロビン・机上チェック・デバッグ・デバッガ・コードカバレッジ・単体テストの自動化
12-4 統合・テスト	ブラックボックステスト・ホワイトボックステスト・限界値分析・スタブ・ドライバ・回帰テスト・信頼度成長曲線・エラー埋込み法・テストケース・同値分割・原因結果グラフ・実験計画法・命令網羅・判定条件網羅・条件網羅・複数条件網羅・トップダウンテスト・ボトムアップテスト・ビッグバンテスト・サンドイッチテスト・システムテスト・負荷テスト・性能テスト・障害回復テスト・ユーザビリティテスト・探索的テスト
12-5 導入・受入れ支援	受入れテスト・移行方式・利用者マニュアル・教育・一斉移行・段階移行・並行運用・パイロット移行・移行リハーサル・検収・運用テスト

まとめ（3/3） この分野で覚える用語

小分類	用語
12-6 保守・廃棄	保守の種類・リバースエンジニアリング・リファクタリング・システムの廃棄・是正保守・予防保守・適応保守・完全化保守・緊急保守・保守性・レガシーシステム・マイグレーション・データ消去